

The Java Virtual Machine (JVM)

JVM is an abstract computing machine that enables a computer to run Java programs. It acts as an intermediary runtime engine that sits between your Java code and the underlying computer hardware and operating system.

The Core Concept: "Write Once, Run Anywhere"

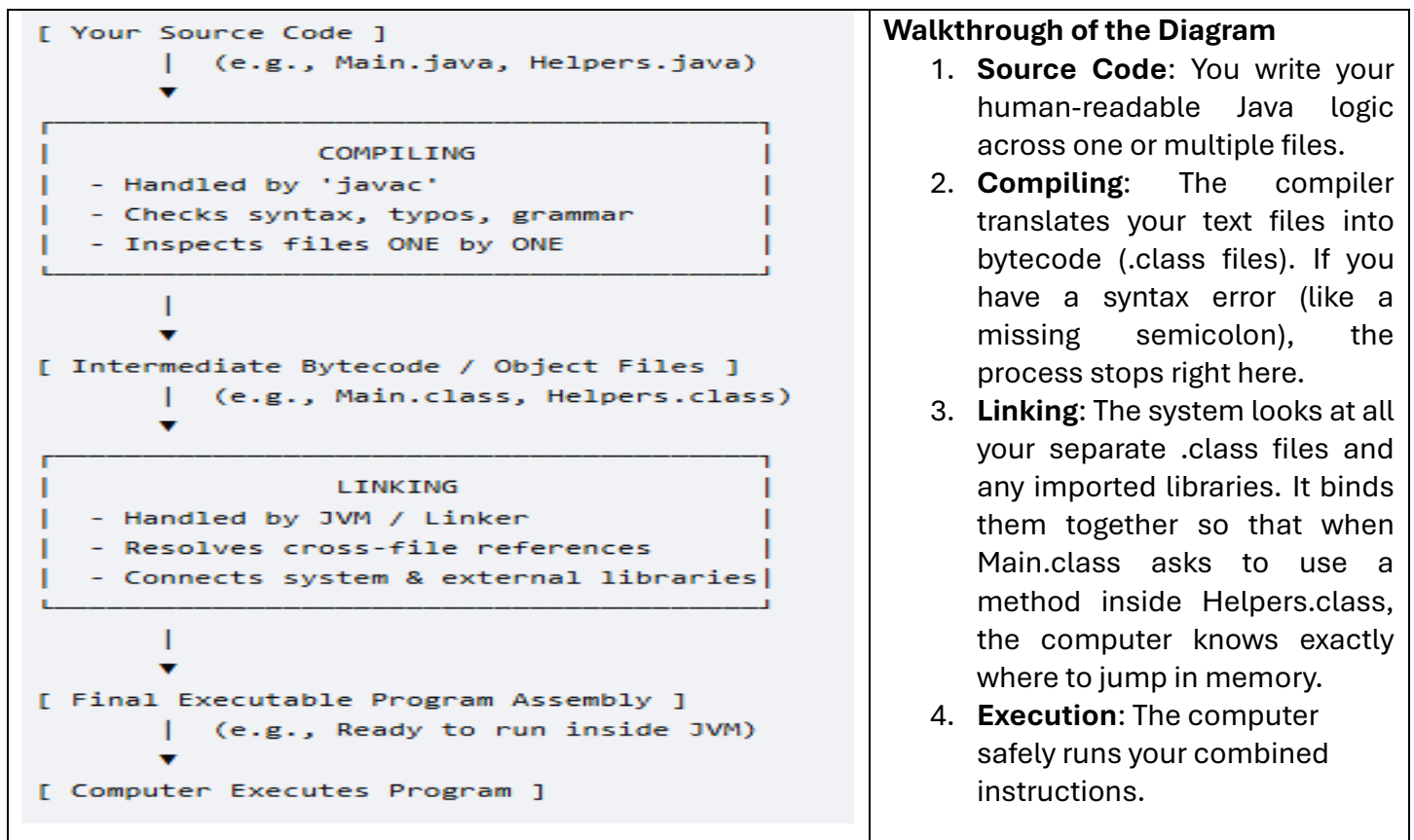
In traditional languages like C++, code is compiled directly into machine instructions tailored to a specific operating system (like Windows or macOS).

Java handles this differently:

1. You write Java source code (.java file).
2. The Java compiler (javac) converts your text into a universal intermediate form called **Bytecode** (.class file).
3. The **JVM** reads this bytecode and translates it on the fly into the specific machine code your current computer understands.

Because every operating system has its own specific version of the JVM, the exact same bytecode file can run on Windows, Mac, Linux, or a smartphone without any changes to the original source code.

Compiling and linking



Compiling and **linking** are the two distinct steps required to transform your human-readable source code into a functional program that a computer can actually execute. While they happen automatically when you run a build command, they perform completely different jobs.

Compiling (Translation)

Compiling is the process of translating your source code text (like a .java or .cpp file) into a lower-level format that machines or virtual machines can understand.

- **What it does:** The compiler acts as a translator. It checks your code for syntax errors, typos, and improper rules. If your code is valid, it translates it.
- **The Output:**
 - In languages like **C/C++**, the compiler generates **Object Code** (individual .obj or .o files containing binary instructions).
 - In **Java**, the compiler (javac) generates universal **Bytecode** (individual .class files).
- **Key Characteristic:** The compiler only looks at **one file at a time**. It translates the file successfully even if that file references functions or modules that live in a completely separate file.

Linking (Assembly)

Linking is the process of gathering all the individual compiled files and external libraries, gluing them together, and building one final, cohesive executable file.

- **What it does:** The linker acts as an assembler. If File A calls a function that is written inside File B, or uses a built-in system library (like Java's System.out.println), the linker goes out, finds the exact memory locations of those missing pieces, and hooks them together.
- **The Output:**
 - In languages like **C/C++**, the linker outputs a single standalone executable file (like an .exe on Windows or a binary file on Mac/Linux).
 - In **Java**, true linking actually happens **dynamically at runtime** by the JVM's Class Loader, though you can manually use a tool like jar to pack your compiled .class files into a single .jar package.
- **Key Characteristic:** The linker looks at the **entire project**. If you try to use a function but forgot to write it or import its library, the compiler will pass it, but the linker will crash with an "unresolved external symbol" error.